Day 8 Evidence Workbooklet — Instructor Key

Expected-vs-Actual Debugging and Test Design

Engineering practice → observed branch outcome → expected branch outcome → likely cause → regression test

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/34		Mastery check	secure / developing / needs support	

The sensor-kit checkout decision logic is now being tested. Some branch outcomes look safe for one kit but fail for another. Today the focus is debugging: separate the observed behavior from the expected behavior, identify a likely cause, choose a minimal fix, and keep a test that would catch the bug again.

Engineering task	Use a repair log to decide whether the checkout decision logic is trustworthy before the lab uses it.
Computational move	Compare expected and actual behavior, connect the mismatch to a code cause, and design tests that cover important branch outcomes.
Python focus	expected branch outcome, actual branch outcome, condition order, Boolean release condition, minimal fix, boundary and regression tests.
Evidence to keep	symptom, expected branch outcome, actual branch outcome, likely cause, minimal repair, test case, and support category.

Day 8 working rules

1. A symptom says what went wrong.
2. Expected behavior says what should have happened.
3. Actual behavior says what the code did.
4. A likely cause points to a condition or order problem.
5. A regression test keeps a known bug from quietly returning.

1 Separate symptom, cause, and repair

recognize debug

The checkout desk finds that some kits are assigned incorrectly. A good debug note separates what happened, what should have happened, and what line likely caused it.

```
charge_percent = 82
calibration_label = "expired"
borrower_initials = "KM"

if charge_percent >= 80:
    action = "release kit"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "hold for review"
```

Pts.	Prompt	My evidence
1	What branch outcome will the current code assign?	**Key:** `release kit`.
1	What branch outcome should the kit receive if calibration is expired?	**Key:** `calibration bench`.
1	Which line lets the kit release too early?	**Key:** The first condition, `if charge_percent >= 80:`, lets any high-charge kit release before checking calibration.
1	Is the first problem a syntax error or a logic error?	**Key:** Logic error. The code runs, but the branch outcome does not match the checkout policy.

2 Build an expected-vs-actual table

trace debug

Use the same code. Complete the table and name the likely cause.

Pts.	Case	Expected branch outcome	Actual branch outcome	Likely cause
2	charge 82, expired calibration	calibration bench	**Key:** see reason	**Key:** Actual branch outcome: `release kit`. Likely cause: the first high-charge condition releases the kit before the calibration check.
2	charge 76, current calibration	charge station	**Key:** see reason	**Key:** Actual branch outcome: `hold for review`. Likely cause: the code has no low-charge branch such as `elif charge_percent < 80`.
2	charge 90, current calibration, blank initials	borrower log needed	**Key:** see reason	**Key:** Actual branch outcome: `release kit`. Likely cause: the first condition checks charge only and reaches release before checking borrower initials.
2	Explain which case best exposes the bug.	**Key:** The charge 82 with expired calibration case is clearest: actual is `release kit`, but expected is `calibration bench`, showing the release condition is too broad.		

3 Design tests before trusting a repair

test explain

A repair is not trustworthy just because it works for one case. Pick cases that cover the branch outcomes.

Pts.	Prompt	My test evidence
2	Give a test case that should produce `"charge station"`.	**Key:** `charge_percent = 76`, `calibration_label = "current"`, and initials present should produce `charge station`.
2	Give a test case that should produce `"calibration bench"`.	**Key:** `charge_percent = 82`, `calibration_label = "expired"`, and initials present should produce `calibration bench`.
2	Give a test case that should produce `"borrower log needed"`.	**Key:** `charge_percent = 82`, `calibration_label = "current"`, and `borrower_initials = ""` should produce `borrower log needed`.
2	Give a test case that should produce `"release kit"`.	**Key:** `charge_percent = 82`, `calibration_label = "current"`, and initials present should produce `release kit`.

Regression-test idea

A regression test is a small case you keep because it catches a bug that already happened once. If the bug comes back, that test should fail again.

4 Choose a minimal fix

implement debug

One safe repair is to require all release evidence before assigning `"release kit"`. Complete the condition.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "hold for review"
```

Pts.	Prompt	My response
2	Why does the release condition need three evidence checks?	**Key:** Release is safe only when charge is high enough, calibration is `current`, and borrower initials are present; one passing check is not enough.
2	Which branch outcome will the expired-calibration case now receive?	**Key:** `calibration bench`.
2	What risk remains if damage evidence is added later but not checked first?	**Key:** A damaged but otherwise ready kit could still be released before the damage rule runs; damage should be checked before release.
2	Name one test you would rerun after this repair.	**Key:** Rerun the expired-calibration regression case: `charge_percent = 82`, `calibration_label = "expired"`, initials present should produce `calibration bench`.

5 Write a repair log

debug test explain

Log field	My repair evidence
Symptom	**Key:** Expired-calibration kits can be assigned to `release kit` when charge is high enough.
Expected behavior	**Key:** A kit with expired calibration should produce `calibration bench`, not release.
Likely cause	**Key:** The first condition checks charge only, so it is too broad and skips later calibration/initials evidence.
Minimal fix	**Key:** Require all release evidence in the release condition: charge at least 80, calibration `current`, and borrower initials present.
Regression test	**Key:** Keep the expired-calibration case: charge 82, calibration `expired`, initials present should produce `calibration bench`.

6 Transfer and support plan

transfer explain

Pts.	Debugging note
4	Explain why expected-vs-actual evidence is stronger than saying "the code is wrong." Use one branch outcome from the sensor-kit checkout example. **Key:** Expected-vs-actual evidence states both the policy and the observed branch outcome. For an expired-calibration kit, expected is `calibration bench` but actual was `release kit`, pointing to the too-broad release condition.
2	Circle the support plan you would use next: symptom / expected branch outcome / actual branch outcome / cause / test case / minimal fix. Name one next action: _____

Bridge to Exam 1 and Day 10

Day 9 is Exam 1. Use expected-vs-actual evidence, minimal repair, boundary tests, and support-plan notes as Exam 1 preparation. Day 10 returns to branch maps and test evidence after the exam and bridges into repeated cases and loops.

VIP Lab Alignment

Bridge Day 8 • Contract 16b4e542994c

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 4 **Activities:** 18.13, 18.14, 18.15, 18.16, 18.17

Use expected-versus-actual evidence on the current sample and transfer cells instead of debugging an obsolete wrapper.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.13 `18-13-work / 18-13-transfer`	**Compare With Tolerance**	Compute absolute distance between two values.
18.14 `18-14-work / 18-14-transfer`	**Count High Readings**	Read a supplied loop without having to design it yet.
18.15 `18-15-work / 18-15-transfer`	**Lamination Fee**	Translate three quantity intervals into ordered branches.
18.16 `18-16-work / 18-16-transfer`	**Temperature Status**	Compare a measurement with a maximum.
18.17 `18-17-work / 18-17-transfer`	**Pickup Window**	Calculate round-trip travel time.

Readiness check before you code

1. Choose one sample or transfer case and record expected result, actual result, likely cause, and the smallest justified repair.

   Answer and explanation: The response must use a real Lab Day 4 cell and distinguish expected result, actual result, cause, and minimal repair; it must not propose rewriting the notebook interface.

2. Name one boundary pair that differs by only one unit or one small measurement change.

   Answer and explanation: Accept a valid adjacent boundary pair tied to the chosen activity, such as 8/9 or 16/17 for an ordered category, or values immediately around a stated tolerance or time boundary.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those studio cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.